系統架構文件（SA）

> **【版本說明】** 本文件為 V6 系統之 **旗艦版 商用級 MVP** 開發時程專屬之客製化規格書。 此版本堅守雙向斷線容錯與基本資安底線，但依據時程嚴格刪減了非必要的自動化服務或進階報表機制。

集中式後台 + 單機本地後台整合版

屬性	內容
版本	v6.2
日期	2026 年 2 月 25 日
適用系統	遊戲平台管理系統
文件類型	系統架構

目錄

1. 文件概述
2. 系統架構概述
3. 集中式後台架構
4. 單機本地後台架構
5. 資料同步與對帳機制
6. 安全性架構
7. 效能設計
8. 部署架構
9. 附錄

---

1. 文件概述

本文件詳細闡述遊戲平台管理系統的整體系統架構，包含集中式後台與單機本地後台兩個層次的架構設計、資料流、通訊機制等內容。

2. 系統架構概述

2.1 雙層後台架構

遊戲平台採用雙層後台架構設計。

- **上層（集中式後台）**：負責全局資料監控、遠端機台管理、全平台交易統計、玩家集中管理等功能。
- **下層（單機本地後台）**：負責本地機台管理、本地遊戲配置、本地交易結算、本地統計分析等功能。 兩個後台相互獨立，但在連線時可進行資料同步與對帳。

2.2 系統組件

系統包含以下主要組件：

- 集中式後台前端（React + Tailwind CSS）
- 集中式後台後端（Express.js + Node.js）
- 單機本地後台前端（React + Tailwind CSS）
- 單機本地後台後端（Express.js + Node.js）
- 集中式資料庫（PostgreSQL）
- 單機本地資料庫（SQLite）
- 遊戲引擎
- 玩家端應用程式

---

3. 集中式後台架構

3.1 前端架構

集中式後台前端採用 **React 19 + Tailwind CSS 4**。使用 Wouter 進行路由管理，shadcn/ui 提供統一的 UI 元件，Recharts 用於資料視覺化，React Context 進行狀態管理。支援響應式設計，適配桌面、平板、手機等多種裝置。

3.2 後端架構

集中式後台後端採用 **Express.js** 框架，Node.js 執行環境。提供 RESTful API 介面，支援 WebSocket 實時通訊。使用 PostgreSQL 資料庫進行資料持久化。

3.3 核心模組

集中式後台包含以下最高規的核心模組：

1. **儀表板模組**：全平台的自訂 Widget 戰情室。
2. **機台心跳深度監測模組**：(旗艦功能) 監控 CPU/RAM 與網路延遲，提早預防當機。
3. **遊戲與版本管理模組**：涵蓋遊戲授權與跨站點維護。
4. **OTA 版本發布中心**：(旗艦功能) 全球遠端熱更新腳本上傳與無縫派發。
5. **玩家管理模組**：VIP 標籤與中央級距錢包。
6. **錢包交易模組**：全域金流流水帳與圖表化分析。
7. **爭議對帳處理專區**：(旗艦功能) 處理斷線異常、人工介入核銷專區與會計除錯。
8. **串接遊戲管理模組**：高度解耦的 PaymentRouter 設定與多家廠商擴展平台。
9. **使用者權限模組**：完全動態自定義的 RBAC 角色系統 (如：審計員、純查帳員)。
10. **介面設定模組**：玩家端與客製化版面的全域配置管理。

---

4. 單機本地後台架構

4.1 前端架構

單機本地後台前端採用與集中式後台相同的（**React 19 + Tailwind CSS 4**），確保設計風格與使用體驗的一致性。支援本地化部署，可在機台上直接運行。

4.2 後端架構

單機本地後台後端採用 **Express.js** 框架，Node.js 執行環境。提供 RESTful API 介面，支援 WebSocket 實時通訊。使用 SQLite 作為本地資料儲存，支援本地快取。

4.3 核心模組

單機本地後台包含以下核心模組：

1. **操作與儀表板模組**：結合開洗分與現場活動捷徑的快搜面板。
2. **熱更新接收器模組**：(旗艦功能) 於背景默默接收中央 OTA 封包、MD5 驗證並自我重啟 (不干擾客人)。
3. **深層機台監控模組**：(旗艦功能) CPU、記憶體佔用實時圖表並回傳。
4. **遊戲配置模組**：本地遊戲啟用/停用。
5. **交易管理與非同步佇列模組**：本地單機流水表與具備 Exponential Backoff 機制的傳輸引擎。

6. **自動對帳系統**：(旗艦強化) 嚴謹地分析本地單據與中央回傳的 Ack 是否完全一致。
7. **機台底層通訊設定**：網路與中心通訊參數、金鑰加密防綁架設定。

---

5. 資料同步與對帳機制

5.1 同步流程

當單機連線到集中式後台時，自動觸發資料同步流程。首先檢查離線期間的本地變更，上傳本地交易紀錄到後台。然後下載後台的遊戲更新、配置更新。最後驗證資料一致性，完成同步。

5.2 衝突解決

當機台離線期間進行的操作與後台操作衝突時，系統具備**雙向重試列隊與會計防護**。系統採用時間戳優先、版本號優先、人工審核爭議專區、交易補償回滾等策略進行解決，確保資料最終一致，絕不憑空銷毀任何記帳斷層。

5.3 對帳機制

系統定期進行對帳檢查，驗證本地資料與後台資料的一致性。對於不一致的項目，記錄並標記為待審核，由管理員手動處理。

---

6. 安全性架構

6.1 認證與授權

系統採用帳號密碼認證，使用 JWT Token 進行會話管理。全面實施基於角色的動態存取控制（Dynamic RBAC），讓集團業主建立完全自定義、顆粒度細緻的操作權限。

6.2 資料加密

所有敏感資料（玩家帳號、交易紀錄、配置資訊）使用 **AES-256** 加密演算法進行加密儲存。通訊採用 HTTPS 加密協議，確保資料傳輸安全。

6.3 交易簽名驗證

所有交易使用 RSA 數位簽名進行驗證，確保交易的真實性與完整性，防止交易被竄改。

6.4 稽核日誌

記錄所有重要操作的詳細日誌，包括操作者、操作時間、操作內容、操作結果等資訊，支援追蹤與審計。

---

7. 效能設計

7.1 效能優化

- 使用資料庫索引優化查詢效能。
- 支援 API 請求的非同步處理，提高系統吞吐量。

---

8. 部署架構

8.1 集中式後台部署

集中式後台部署在雲端或資料中心。前端與後端服務皆使用容器化部署（Docker），支援自動擴展。資料庫使用主從複製實現高可用。

8.2 單機本地後台部署

單機本地後台與遊戲引擎部署在同一台機器上。支援 Windows、Linux、macOS 等作業系統。使用 Node.js 執行環境，SQLite 作為本地資料庫。

8.3 網路拓撲

- 集中式後台與單機本地後台通過 Internet 連線。
- 單機本地後台與遊戲引擎通過本地網路連線。
- 玩家端應用程式與遊戲引擎通過本地網路連線。

---

9. 附錄

9.1 技術總結

層次	技術	說明
前端	React 19 + Tailwind CSS 4	統一的前端技術
後端	Express.js + Node.js	輕量級後端框架
集中式資料庫	PostgreSQL	關聯式資料庫
單機本地資料庫	SQLite	本地資料儲存
通訊	HTTP/HTTPS + WebSocket	實時通訊
加密	AES-256 + RSA	資料安全